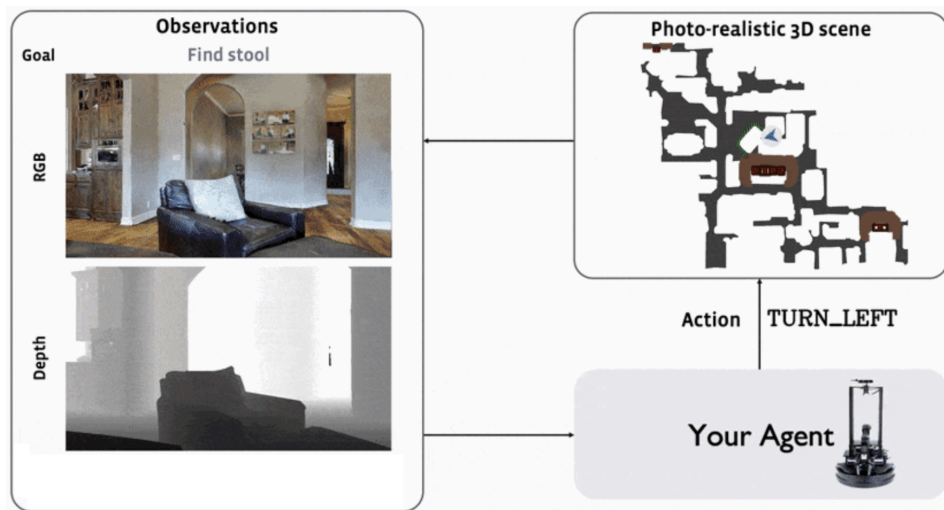


AI Capstone HW2

3D Scene Reconstruction

Announce: 3/26, Deadline: 4/9(Thursday) 23:59

Introduction



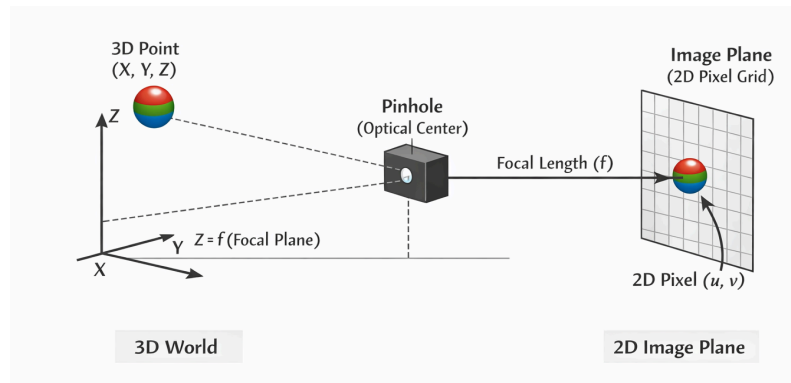
Habitat Navigation Challenge 2022 [1]

Imagine that you are building a **household robot** that can move around a home. Before it can perform tasks such as finding objects, cleaning rooms, or assisting people, it must first **understand the structure of its environment** so that it knows where things are and how to move safely and efficiently. For example, if you ask the robot to get something from the fridge, it needs to know where the kitchen is and how to navigate there without colliding with the dining table.

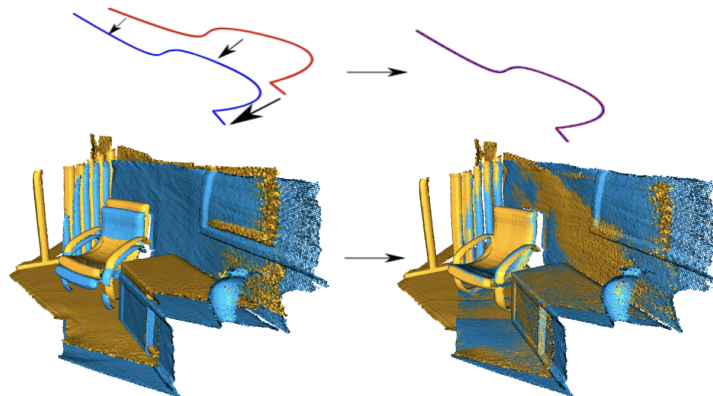
In this homework, you will take on the role of this robot and **reconstruct a 3D map of an apartment** from raw RGB-D data. You will operate an agent in the `apartment_0` scene from the Replica dataset [2] using Habitat-Sim, collecting observations, converting them into 3D point clouds, and aligning them into a consistent global scene. With this map, the robot can navigate the environment and perform everyday tasks. By completing this homework, you will gain hands-on experience in:

- **Data Collection:** You will control an agent to explore an environment and collect RGB-D data. How should the robot move to cover the entire space efficiently?

- **3D Reconstruction:** You will convert depth images into 3D point clouds to represent the scene. Through this process, you will gain an intuitive understanding of the **pinhole camera model** [3] and how depth values are projected into 3D space.



- **Scene Alignment:** You will align multiple point clouds with **Random Sample Consensus (RANSAC)** [4] and **Iterative closest point (ICP) algorithm** [5] to build a global map, combining observations from different viewpoints into a unified representation.



- **Evaluation:** You will evaluate the quality of your reconstruction by comparing estimated and ground truth trajectories, and analyze how different choices affect the final result.

While this assignment is conducted in a virtual environment, it serves as a starting point for considering the challenges involved in building and using such maps for real-world robotic tasks.

GitHub repository:

<https://github.com/HCIS-Lab/ai-capstone26/tree/main/hw2>

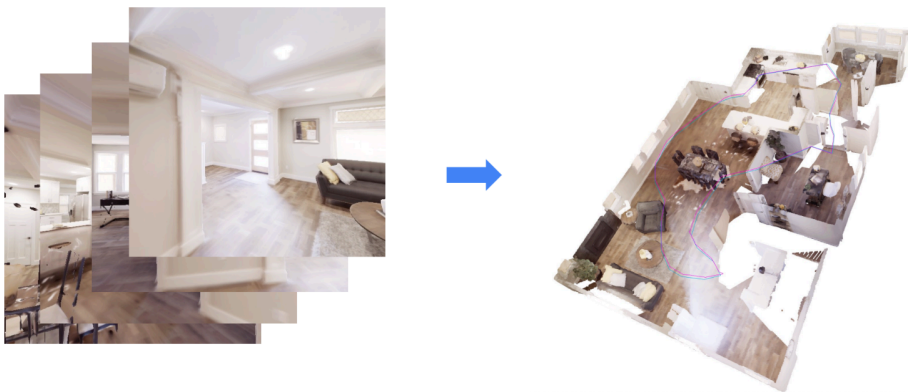
Implementation

We provide code templates: `load.py` for data collection and `reconstruct.py` for implementing reconstruction algorithms, please check [hw2 readme](#) on github. You will need to complete the sections marked with #TODO.

Task 1: Data Collection

- **Dataset Download:** Download the `apartment_0` environment from the [Replica Dataset](#).
- **Data Collection:** Control the agent to explore the **first floor and second floor of apartment_0**. Save the RGB images, depth images, and the camera ground truth poses as `GT_pose.npy`.

Task 2: ICP Alignment & 3D Map Reconstruction



- **Pipeline:** Unproject depth images → Voxelization (Downsampling) → Global Registration (RANSAC) → Local Registration (ICP).
 1. **Unprojection:** Convert depth images into 3D point clouds.
 - Camera Model: Pinhole (Resolution 512x512).
 - FOV: Horizontal and Vertical are both 90 degrees.
 - Depth_scale = 1000
 - *Note: You should **not use Open3D functions here**; instead, you should implement each step yourself.*
 2. **Voxelization (Downsampling):** Reduce the number of points to improve computational efficiency.
 3. **Global Registration (RANSAC):** Estimate an initial alignment between point clouds to handle large viewpoint differences.

4. **Local Registration (ICP):** Refine the alignment by minimizing the distance between corresponding points.

- **Required and Bonus Tasks:**

1. **Open3D Version (Required):** Use Open3D's built-in registration function (RANSAC, voxel down, Point-to-Point/Plane ICP) [6].
2. **My ICP Version (Bonus):** Implement your own ICP algorithm, including nearest neighbor search and transformation estimation.

*Note: Your own ICP should **complete within 5 minutes during the demo**; otherwise, you will not receive the bonus points.*

Task 3: Camera Pose Estimation & Visualization

- **Scene:** Accumulate point clouds into a 3D scene and **remove the ceiling** for better observation.
- **Trajectory:** Visualize both the **Estimated Trajectory (Red)** and the **Ground Truth Trajectory (Black)**.
- **Evaluation:** Calculate the **Mean L2 Distance** between estimated camera positions (x,y,z) and ground truth positions (x,y,z).

Example results

Red line: The estimated camera pose

Black line: The ground truth camera pose (GT_pose.npy)

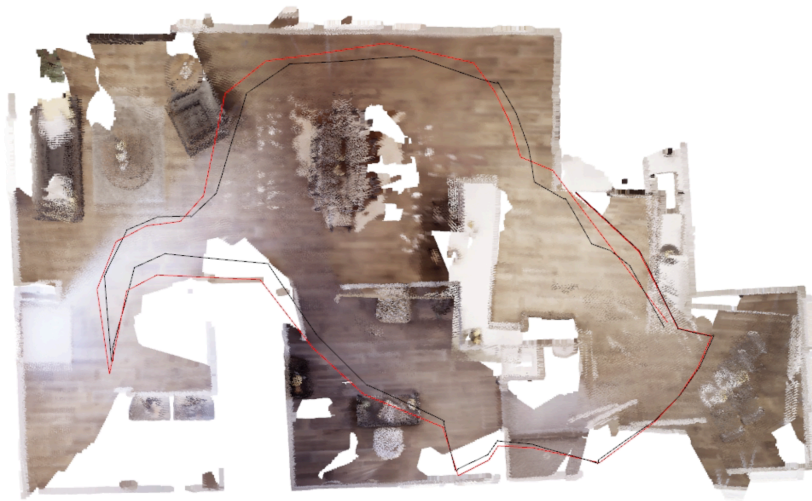
Note: the below results are the screenshots.

- **The first floor of apartment_0**

Data size = 152



Data size = 129



The above two results show the **importance of the data collection step**. Although collecting data in the same scene, with different data size, trajectory, distance to the object, the reconstructed result differs a lot.

- **The second floor of apartment_0**

Data size = 213

